

Curvine 系统扩展方案

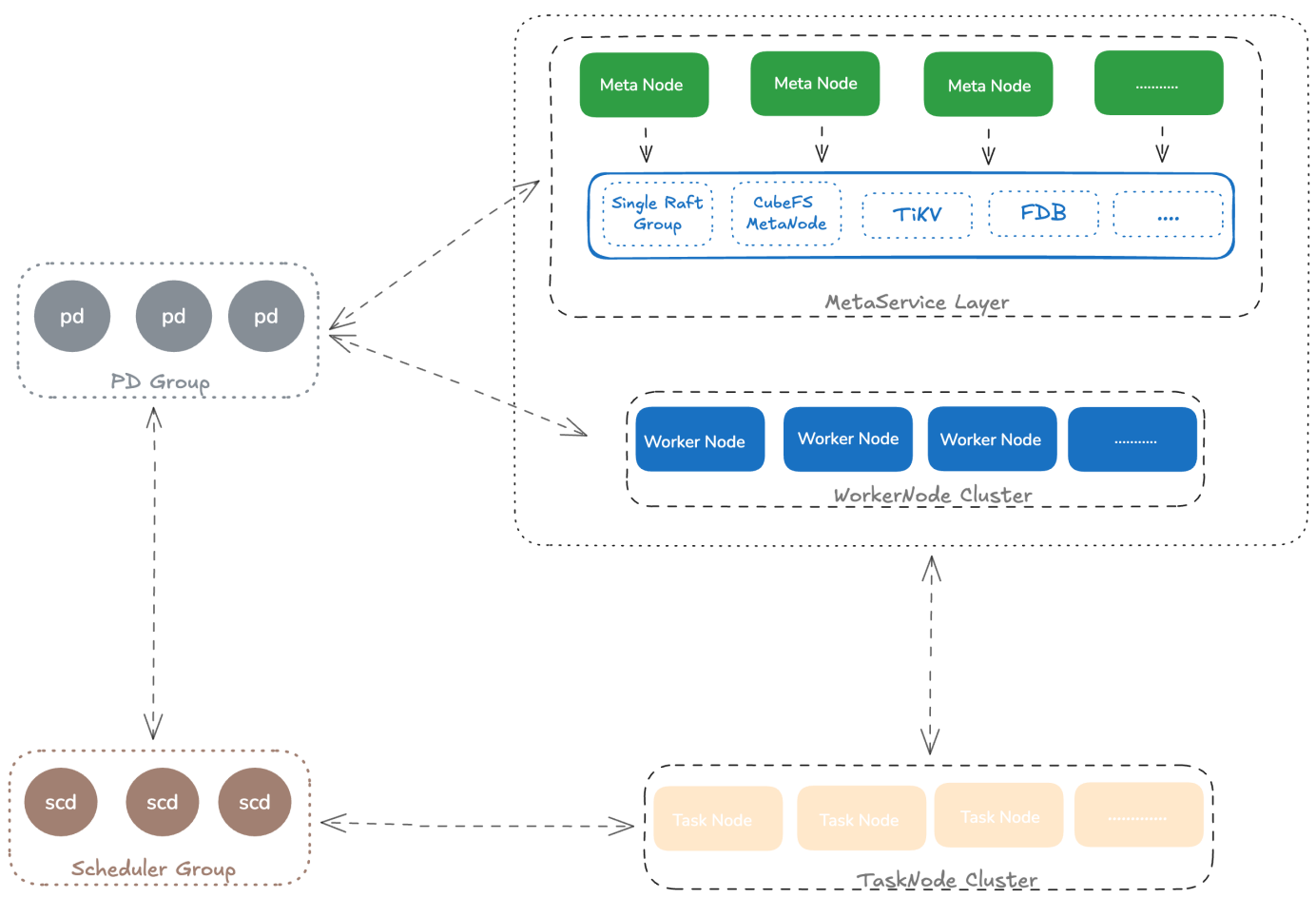
一、背景

Curvine 是一个高性能、云原生的分布式缓存系统，专为现代数据密集型应用设计。但落地仍面临一些挑战：

- 海量元数据管理：需要支持百亿文件缓存，数十PB到百PB数据量规模。
- 大量小随机读请求：应对 BI 全表扫描和索引扫描，AI 训练索引文件加载场景。
- 持续的大量预热任务请求：避免对在线读写产生影响。
- 高数据可靠性保障：缓存的数据避免节点故障导致的丢失。

二、系统设计

2.1 整体架构



模块	主要功能	关键实现
PD (Placement Driver)	集群的"大脑", 存储和管理集群元信息, 节点的管理和调度, 主要存储如下元数据: 全局配置信息、挂载表信息、MetaNode 节点信息和拓扑结构、WorkerNode 节点信息和拓扑结构	参考 TIKV PD 实现, 使用 raft 保障高可用和数据强一致性, rocksdb 存储元数据, 复用现在已有的代码模块
WorkerNode	数据块的存储和缓存管理, 基于 block 基础切分细粒度 page 存储, 应对海量随机读场景	基于现有 worker 基础上进行优化改造
MetaService Layer	元数据层, 用于存储海量文件和目录的元数据信息以及文件 block 信息等; 设计为一个抽象层, 通过接口化支持可扩展的 KV 存储引擎, 方便复用成熟的分布式 KV 系统(比如: FDB, TIKV等)	MetaNode 设计为无状态服务 (除 embed single raft group), 启动后自动注册到 PD 中, MetaNode 需要配置使用的具体的 kv 引擎, 热插拔设计, 根据需求适配不同的 KV 引擎
Scheduler	集群任务管理和调度中心, 用于管理各种异步任务, 包括管理员手动任务, 集群自动调度任务, 业务预热/降冷任务等; 任务的具体执行分发到 TaskNode 执行	使用 Raft 保障高可用, 使用 rocksdb 存储任务 id和相关的运行状态信息, 提供 RESTFULL API, 方便与管理平台等打通联动
TaskNode	异步任务的执行者, 执行 Scheduler 下发的各类任务, 并将执行结果汇报给 Scheduler	无状态服务, 任务自动拆分多个 step, 使用流水线异步执行

2.2 元数据层

为了解决元数据扩展能力, 支持百亿元数据规模, 考虑引入分布式架构的元数据服务。目前业界有三种做法:

1. 实现子树的 sharding 分区方案, 基于 sharding + raft + rocksdb 来做元数据管理。
2. 存算分离架构, 使用 stateless server + Distributed KV/DB来做元数据管理。
3. 联邦架构(Federation), 基于目录树拆分多个集群, 每个集群独立负责自己的元数据管理。

这三种实现方式的对比如下:

维度	当前架构（RocksDB + Raft）	联邦（Federation）架构	基于分布式KV (FoundationDB/TiKV等)
状态管理	有状态（每个Master 维护本地RocksDB）	有状态（每个 Zone/Master 管理本地命名空间）	无状态（所有状态在分布式KV中）
水平扩展	困难（需要重新分片 RocksDB）	较容易（新增 Zone 或集群即可扩展）	轻松（直接加MetaServer实例）
高可用	需要自己实现Raft选主	每个 Zone 独立高可用，全局多活可配置	自动（分布式KV自带）
事务支持	需要自己实现（通过 Batch）	仅限单 Zone 内事务	原生支持ACID事务
跨分片事务	需要自己实现2PC	不支持或有限支持（需外部协调）	支持（FDB/TiKV原生支持）
一致性	强一致（基于Raft同步）	强一致（Zone 内强一致，Zone 间最终一致）	强一致（Snapshot Isolation）
运维复杂度	需要管理RocksDB快照、日志清理	中等偏高（需管理多个 Zone 的一致性与同步）	简单（分布式KV自带运维工具）
故障恢复	需要切主+日志回放	快速（Zone 级别独立恢复）	即时（其他MetaServer继续服务）
性能	数据存储本地 RocksDB，访问延迟较小	总体性能较好，但跨 Zone 访问存在延迟	访问TIKV/FoundationDB，基于MultiGet等接口，会多一跳网络。可以引入缓存缓解

方案实现的难易程度上，Federation架构更容易实现；从扩展性的角度看，使用存算分离架构更加适合更大规模的增长。未来我们在AP和训练场景，常驻集群的文件数都可能会达到数十亿、上百亿，如果只使用Federation架构，集群的数量过多，难以维护。为了满足未来日益增长的需求，首先要提升单集群的服务上限。

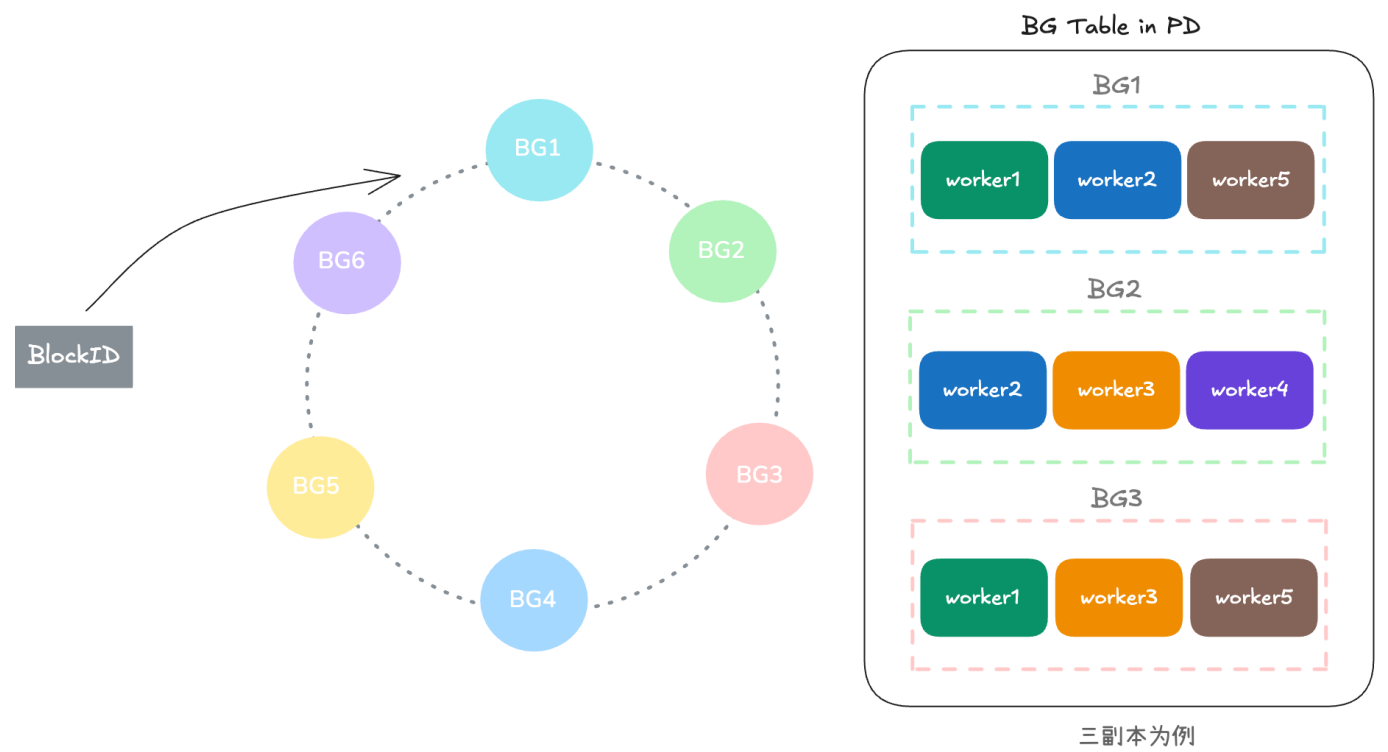
根据对业界实现方案的调研，基于单 Raft 组 rocksdb 方案在深度优化下，可以实现百亿级别的元数据规模（block 元数据下放到存储节点），单节点 QPS 可达到 50W(文件基于细粒度的锁)。因此在保持架构支持高扩展的前提下，重点优化单 Raft 组的元数据能力，提高其性能。

2.3 数据层

元数据架构改造后可支持百亿级别，数据量规模也会随着线性增长，block 元数据管理也会面临一些挑战，当前架构中 block 信息主要有如下索引信息：

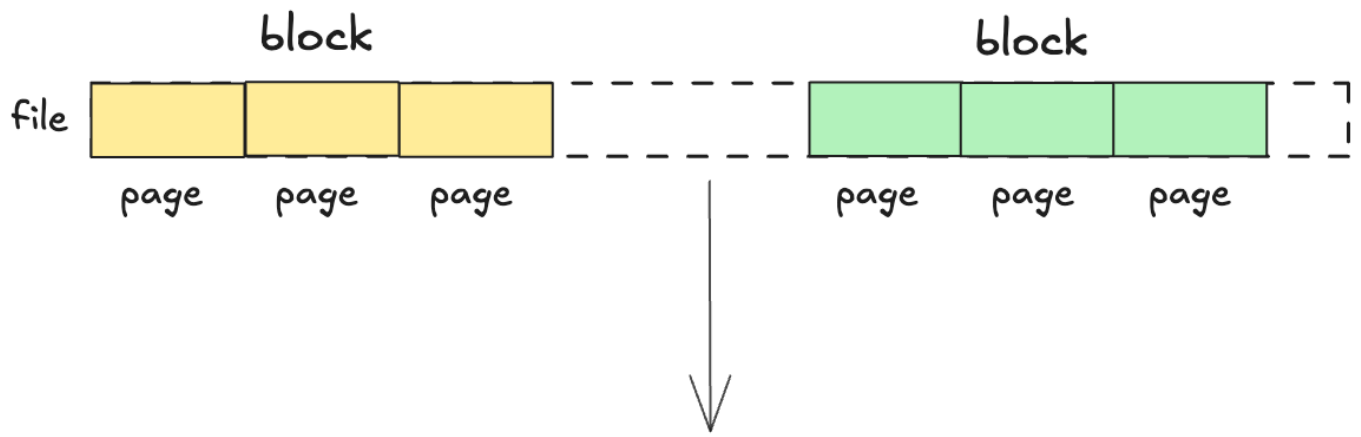
- File -> Block List
- Block -> Worker Node List
- Worker -> Block List (反向索引，当 worker 节点故障后，可快速发现丢失的 block 信息，进行副本复制)

元数据改成分布式 KV 后，File -> Block list 的索引信息仍需要存储，后面两个元数据存储成本较高，尤其是 Worker -> Block list 的映射。为了解决该问题并尽可能减少元数据量，引入 BlockGroup (BG) 概念，BlockGroup 用来管理一组 block。BlockGroup 和 Worker 节点为多对多关系，BlockGroup 支持存储类型设置、副本数设置，每个 Worker 会包含多个 BG，数据迁移和复制以 BG 为单元。BG 的信息存储在 PD 中，由 PD 负责创建和管理（负载均衡）。BlockGroup 大小可配置，以 128GB 为例，100PB 的缓存数据，共需要 819200 个 BG，单个 BG 元数据一般在 200 字节内(和副本数量有关)，总共需要 ~200MB，因此 BlockGroup 管理不会存在瓶颈，即使全内存也能够放得下。如果 BlockGroup 不限制容量大小，就可以和一致性 Hash 方式无缝结合。BlockGroup 变为 Hash 环中的一个节点，Block 基于 Hash 规则直接映射到 BlockGroup，索引数据中 2 和 3 部分就完全不用存储，大大减少了元数据量（基于固定的 block 大小，第一部分的元数据也可以省略），因此通过 BlockGroup 能够带来较好的灵活性。



针对海量随机读场景，进一步将 block 切分为 page (大小可配置，比如 4MB)，因此文件实际按照 page 切块存储。细粒度的 page 切块，对于内核 readahead 和 page cache 友好，有利于提高随机读的性能。为什么引入 page，不直接将 block 按照细粒度的块切分呢？主要平衡元数据量，对于中大文件如果按照细粒度的块切分，需要记录大量 block 信息，元数据压力将倍增；这里文件元数据还是按照大块 block 来记录，worker 存储时按照 page 切块存储，读取时根据 offset 定位对应的 page。对于顺序读 client 使用协程进行并发 page 预读，也能够带来一定的性能提升。如果 BG 采用一致性 Hash 的实现方式，block 可以直接按照 4MB 数据来存储，数据块会更加分散，能充分利用集群的吞吐能力。

为了保障缓存数据的可靠性，worker 节点除了存储 block 块信息外，也需要记录 block 的元数据，比如使用 rocksdb 来存储，当节点出现重启或故障时，可以利用元数据定位丢失的 block 数据，进行快速恢复，保障数据的可靠性（缓存命中率），对于一些长生命周期的缓存文件，也可以做数据一致性校验，方便对未来一些功能的扩展。



BG 使用方式	简介	优点	缺点
有容量大小的 BG	BG 有容量限制，当BG 存储量或block 数量达到一定量级后，则不可写入，PD 创建新的 BG 并分配到 Worker 节点上	节点扩缩容友好，可以有效控制 BG 迁移的数据量，副本数量比较灵活，可以灵活满足不同目录对副本数量的需求，BG 不足时自动创建	管理相对复杂一些，需要动态扩缩 BG 数量，调节权重等等，需要记录 file block 信息，以及 block 所属的 BG，元数据量会有较多的占用
无容量的 BG(一致性 HASH 方式)	BG 容量不做限制，集群创建好后，BG数量相对固定	block 元数据完全不用存储，基于 inode hash 即可，有利于元数据服务深度优化，管理相对方便，BG 数量固定	扩缩不够友好，Hash 环变动，必然引起部分节点的 block 失效，副本数量需要一致，灵活性差

2.4 异步任务层

为了应对用户海量预热任务，避免 Master-Worker 架构下主节点任务过多影响在线请求等问题，单独拆分为两个模块来完成该功能，方便未来的功能扩展。

- **Scheduler:** 作为任务调度服务，负责生成任务 ID 并维护任务的执行状态信息，同时维护 TaskNode 的节点信息和拓扑结构。当收到任务请求后生成唯一的任务 ID并返回给客户端，根据 TaskNode 的拓扑结构使用一致性 Hash 分发给 TaskNode 执行。Scheduler 采用 Raft 协议保障高可用，整体处理逻辑都比较轻量，避免成为瓶颈。Scheduler 会针对不同优先级任务类型做单独的队列处理，避免相互影响。
- **TaskNode:** 负责任务的具体执行，并定期将任务执行状态汇报给 Scheduler 节点，当 Scheduler 节点长期未收到任务状态更新时，会重新分发任务，保障任务执行的可靠性。TaskNode 为无状态服务，方便扩缩容，服务启动后会定期向 Scheduler 节点进行注册，单个节点故障仅影响部分任务执行耗时。为了避免产生大量任务的状态更新请求，TaskNode 会采用批量更新的方式。